南京大学
NANJING UNIVERSITY

数据结构
Data Structure

许 蕾

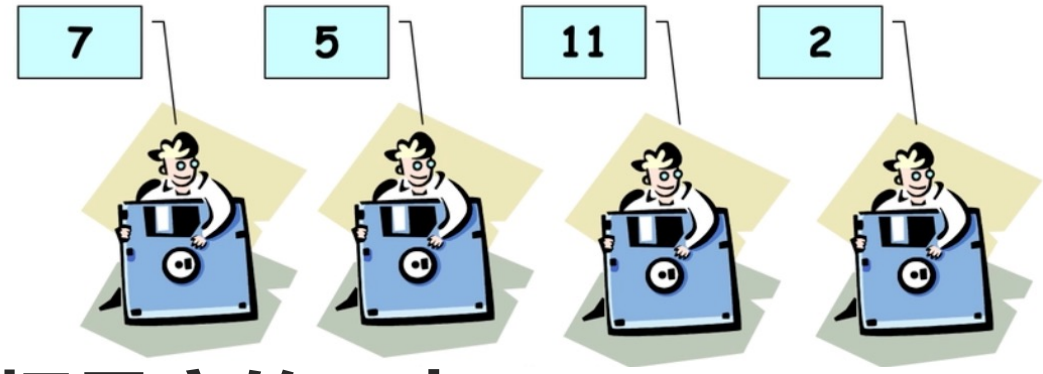
xlei@nju.edu.cn

南京大学计算机学院

第五章 树与二叉树

- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树

5.8 堆（Heap）

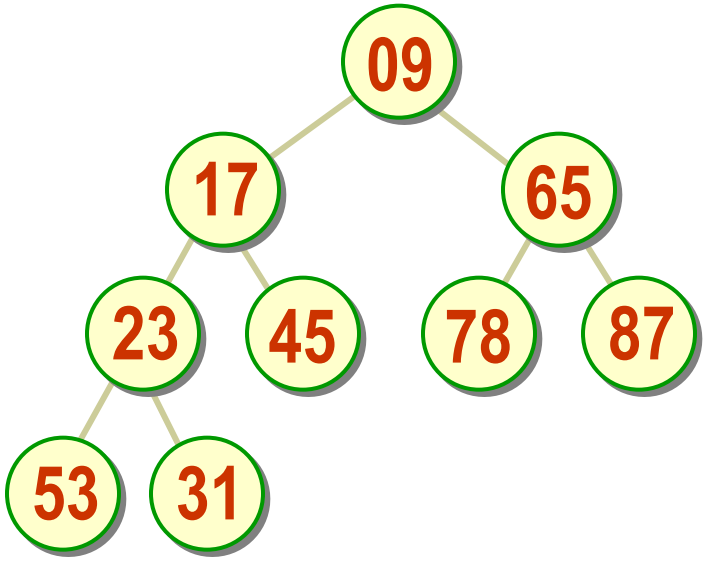


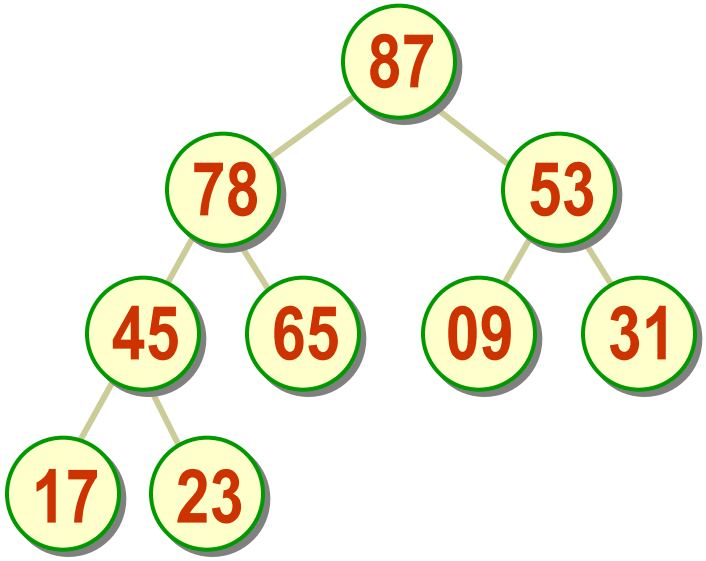
优先级队列

- **每次出队列的是优先权最高的元素。**
- **用堆实现其存储表示，能够高效运作。**

```
template <class E>
class MinPQ {          //最小优先级队列类的定义
public:
    Virtual bool Insert (E& x) = 0;
    Virtual bool Remove (E& x) = 0;
};
```

5.8.1 堆的定义





完全二叉树顺序表示

$K_i \leqslant K_{2i+1}$ && $K_i \leq K_{2i+2}$

最小堆

完全二叉树顺序表示

$K_i \geqslant K_{2i+1}$ && $K_i \geq K_{2i+2}$

最大堆

堆的元素下标计算

- 由于堆存储在下标从 0 开始计数的一维数组中，因此在堆中给定下标为 i 的结点时
  a) 如果 $i = 0$，结点 i 是根结点，无双亲；否则结点 i 的父结点为结点 $\lfloor (i-1)/2 \rfloor$；
  b) 如果 $2i+1 > n-1$，则结点 i 无左子女；否则结点 i 的左子女为结点 $2i+1$；
  c) 如果 $2i+2 > n-1$，则结点 i 无右子女；否则结点 i 的右子女为结点 $2i+2$。

最小堆的类定义

```
template <class E>
class MinHeap : public MinPQ<E> {
                        //最小堆继承了（最小）优先级队列
public:
    MinHeap (int sz = DefaultSize);     //构造函数
    MinHeap (E arr[], int n);           //构造函数
    ～MinHeap() { delete [ ] heap; }    //析构函数
    bool Insert (E& x);                 //插入
    bool Remove (E& x);                 //删除
```

```
    bool IsEmpty () const          //判堆空否
      { return  currentSize == 0; }
    bool IsFull () const           //判堆满否
       { return currentSize == maxHeapSize; }
     void MakeEmpty () { currentSize = 0; }  //置空堆
private:
    E *heap;                       //最小堆元素存储数组
    int currentSize;               //最小堆当前元素个数
    int maxHeapSize;               //最小堆最大容量
    void siftDown (int start, int m);     //向下调整算法
    void siftUp (int start);               //向上调整算法
};
```

5.8.2 堆的建立

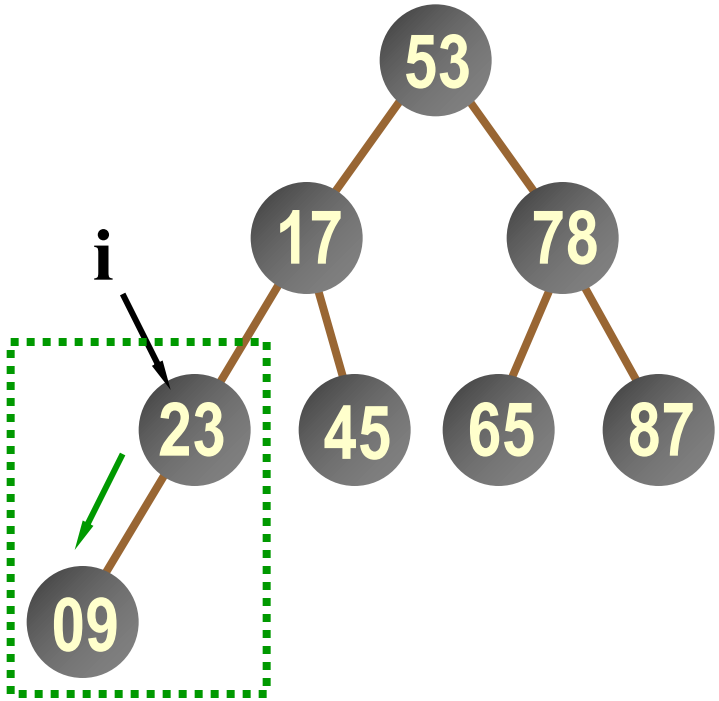
方式一

```
template <class E>
MinHeap<E>::MinHeap (int sz) {
    maxHeapSize = (DefaultSize < sz) ?
                                  sz : DefaultSize;
    heap = new E[maxHeapSize];          //创建堆空间
    if (heap == NULL) {
        cerr << "堆存储分配失败！" << endl; exit(1);
    }
    currentSize = 0;                    //建立当前大小
};
```

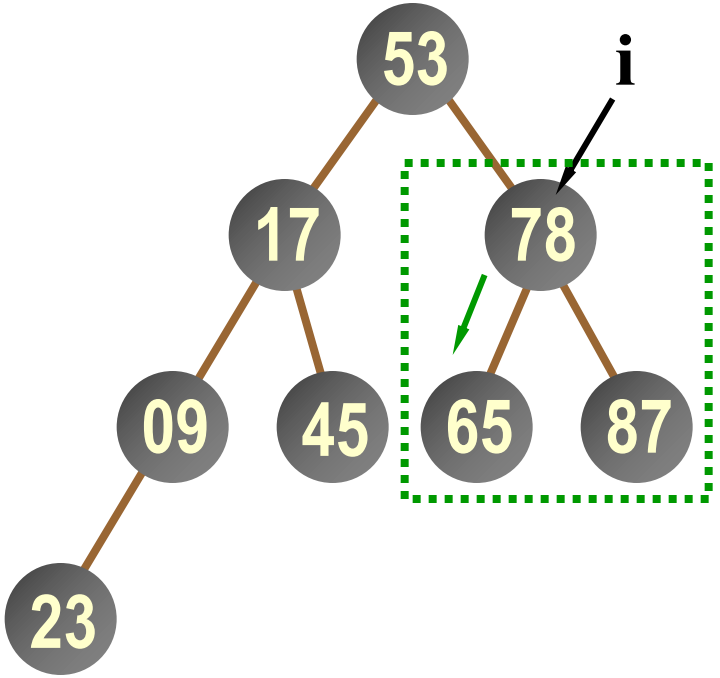
方式二

```
template <class E>
MinHeap<E>::MinHeap (E arr[], int n) {
    maxHeapSize = (DefaultSize < n) ? n : DefaultSize;
    heap = new E[maxHeapSize];
      if (heap == NULL) {
      cerr << "堆存储分配失败！"  << endl;  exit(1);
    }
    for (int i = 0; i < n; i++) heap[i] = arr[i];
    currentSize = n;                    //复制堆数组，建立当前大小
      int currentPos = (currentSize-2)/2;
                                  //找最初调整位置：最后分支结点
    while (currentPos >= 0) {                        //逐步向上扩大堆
        siftDown (currentPos, currentSize-1);  //局部自上向下下滑调整
        currentPos--;
    }
};
```

将一组用数组存放的任意数据调整成堆

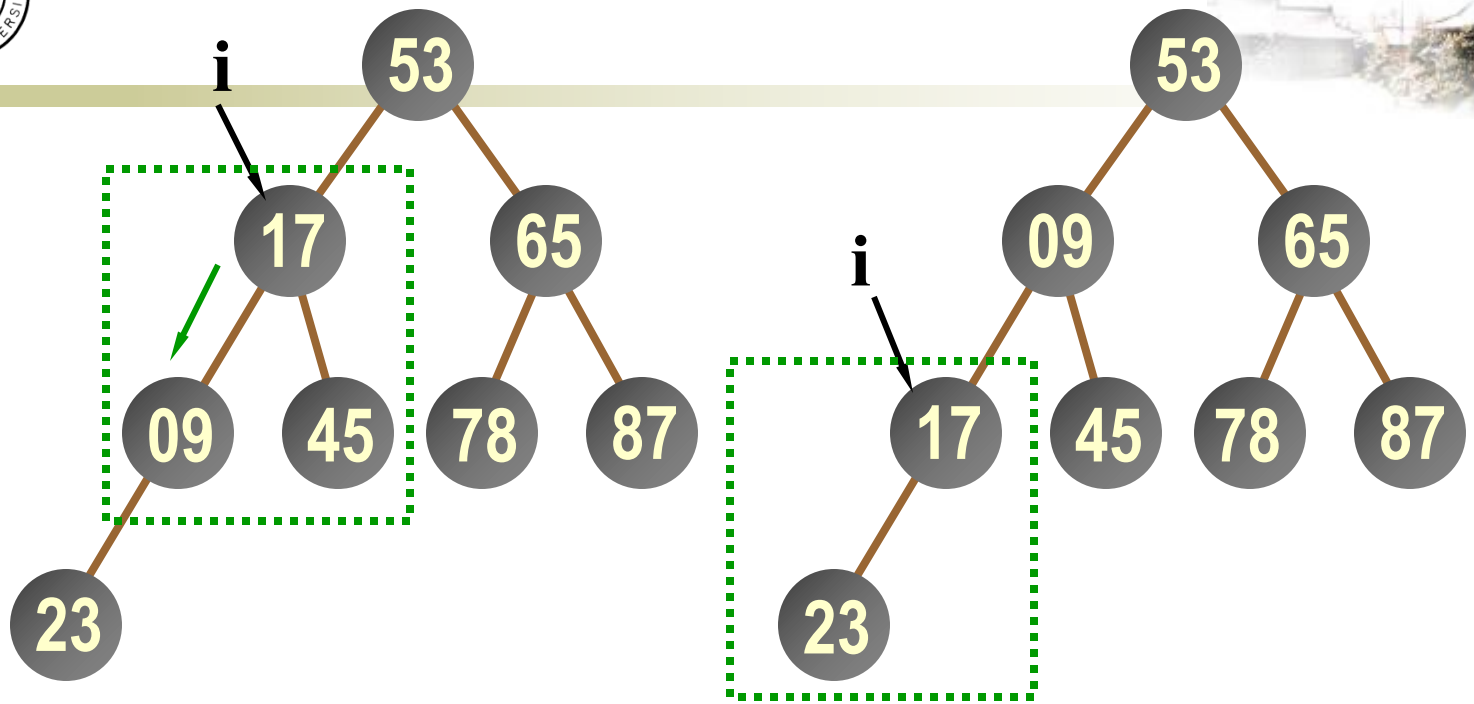


currentPos = i = 3



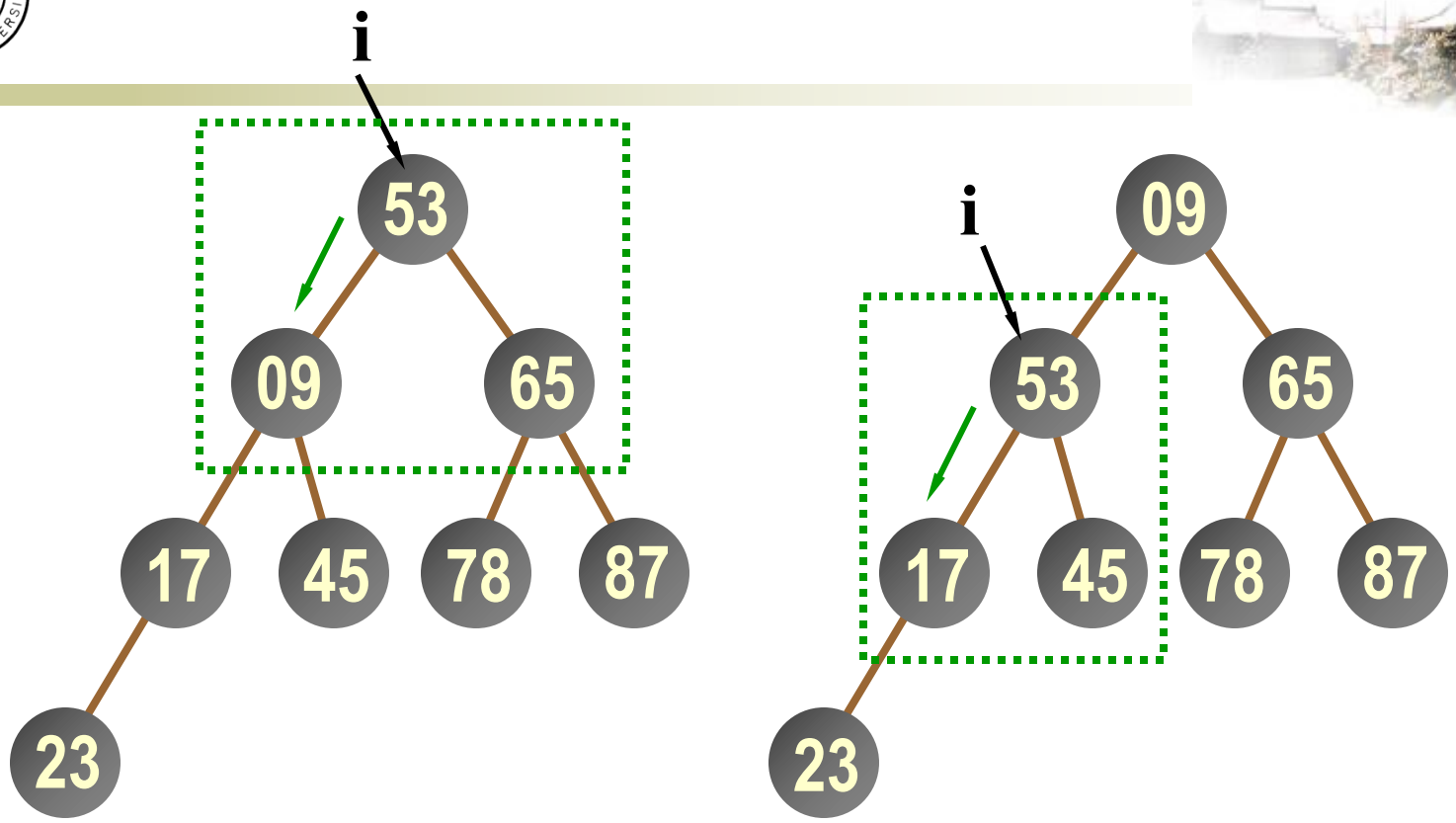
currentPos = i = 2

自下向上逐步调整为最小堆



currentPos = i = 1

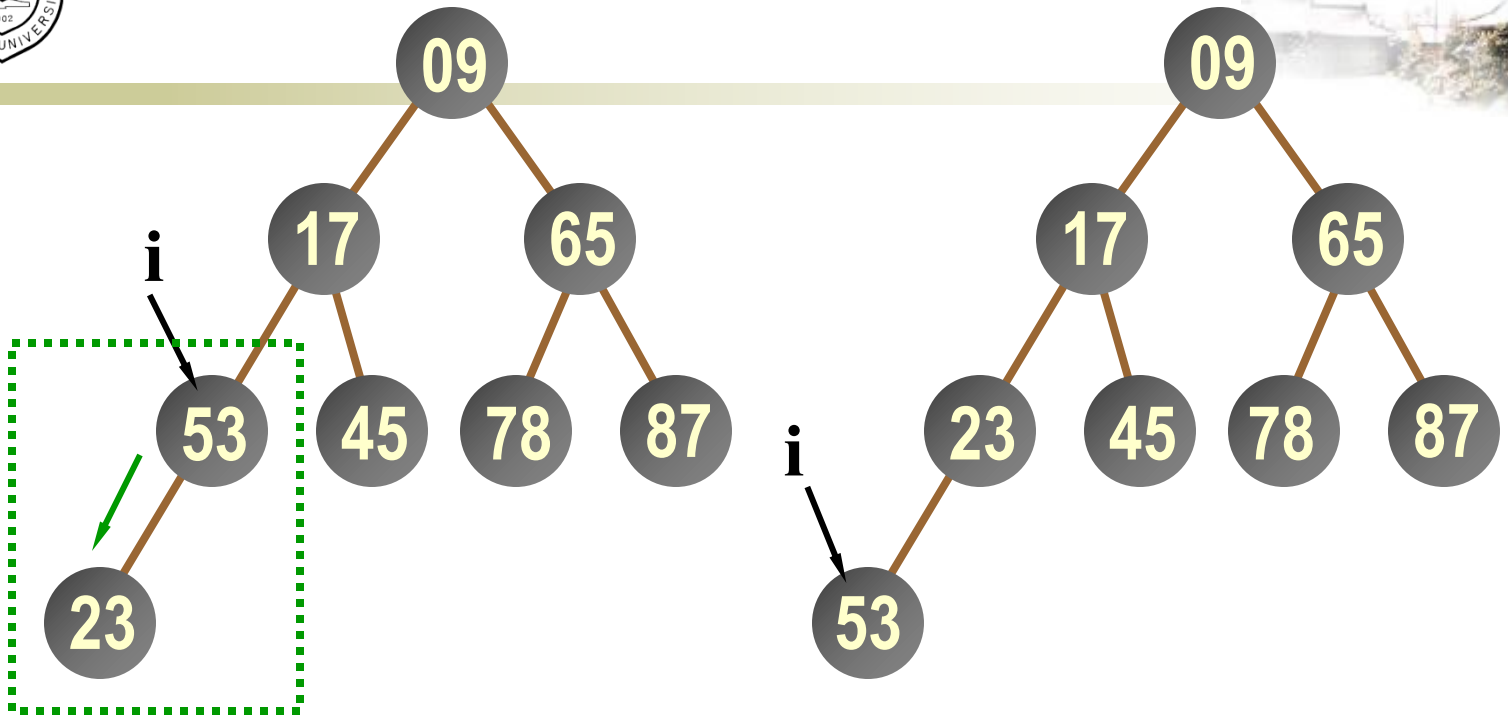
Step 1 **Step 2**



currentPos = i = 0

Step 1

Step 2



currentPos = i = 0

Step 3 **Step 4**

最小堆的下滑调整算法

```
template <class E>
void MinHeap<E>::siftDown (int start, int m ) {
//私有函数: 从结点start开始到m为止, 自上向下比较,
//如果子女的值小于父结点的值, 则关键码小的上浮,
//继续向下层比较, 将一个集合局部调整为最小堆。
    int i = start,   j = 2*i+1;          //j是i的左子女位置
    E temp = heap[i];
    while (j <= m) {                     //检查是否到最后位置
        if ( j < m && heap[j] > heap[j+1] ) j++;
                                         //让j指向两子女中的小者
```

```
        if ( temp <= heap[j] ) break;      //小则不做调整
        else { heap[i] = heap[j];  i = j; j = 2*j+1; }
                                           //否则小者上移, i, j下降
    }
    heap[i] = temp;                        //回放temp中暂存的元素
};
```

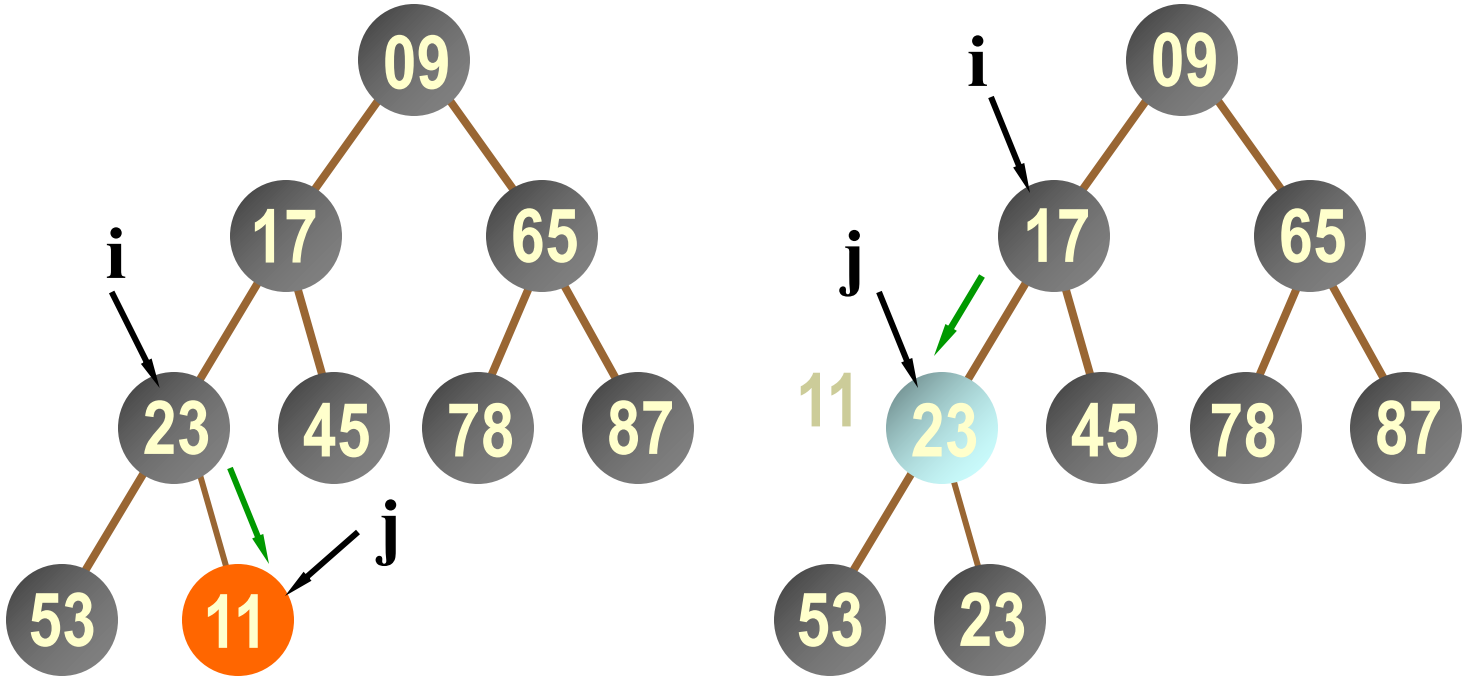
5.8.3 最小堆的插入

- 每次插入都加在堆的最后，再自下向上执行调整，使之重新形成堆，时间复杂度为$O(\log_2 n)$。

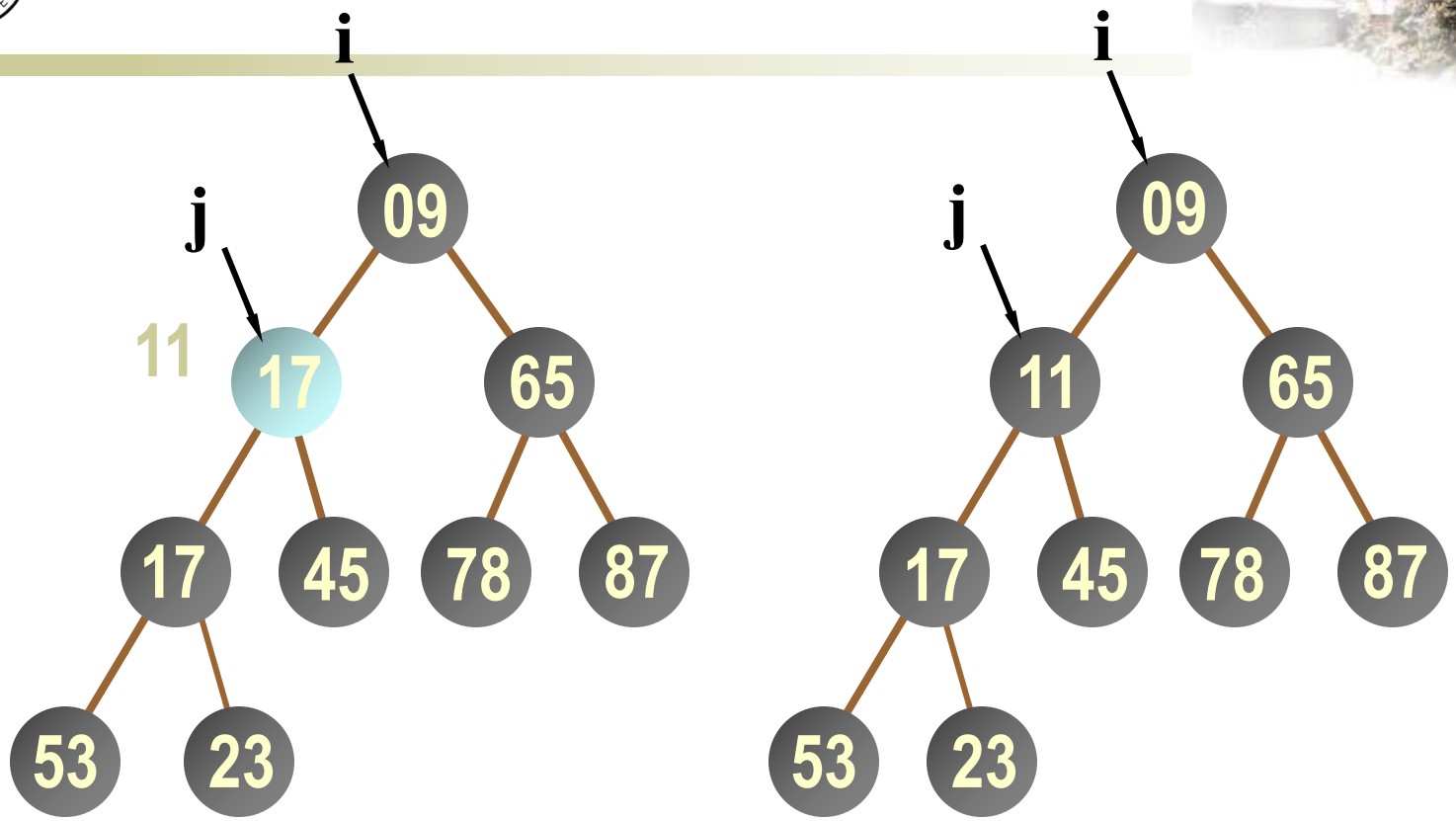
```
template <class E>
bool MinHeap<E>::Insert (const E& x ) {
//公共函数: 将x插入到最小堆中
    if ( currentSize == maxHeapSize )          //堆满
      { cerr << "Heap Full" << endl;  return false; }
    heap[currentSize] = x;                      //插入
    siftUp (currentSize);                       //向上调整
    currentSize++;                              //堆计数加1
    return true;
};
```

```
template <class E>
void MinHeap<E>::siftUp (int start) {
//私有函数: 从结点start开始到结点0为止, 自下向上比较,
// 如果子女的值小于父结点的值, 则相互交换, 这样将
//集合重新调整为最小堆。关键码比较符<=在E中定义。
    int j = start,  i = (j-1)/2;   E temp = heap[j];
    while (j > 0) {                //沿父结点路径向上直达根
        if (heap[i] <= temp) break;
                                   //父结点值小, 不调整
        else { heap[j] = heap[i];  j = i;  i = (i-1)/2; }
                                   //父结点值大, 调整
    }
    heap[j] = temp;                //回送
};
```

最小堆的向上调整

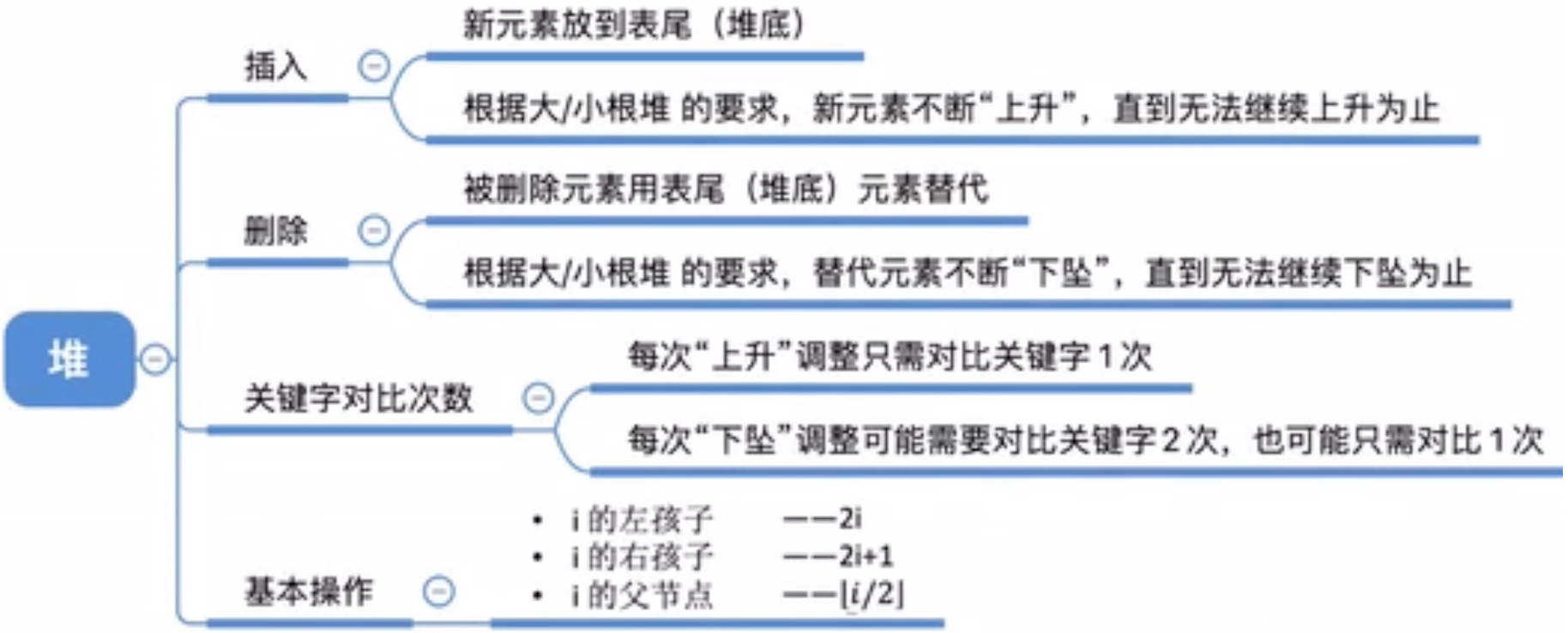


在堆中插入新元素11


i
j
09
11
17
65
17
45
78
87
53
23
i
j
09
11
65
17
45
78
87
53
23

5.8.3 最小堆的删除算法

```
template <class E>
bool MinHeap<E>::Remove (E& x) {
    if ( !currentSize ) {                    //堆空, 返回false
        cout << "Heap empty" << endl;  return false;
    }
    x = heap[0];                             //删除0号元素
    heap[0] = heap[currentSize-1];  //最后元素填补到根结点
    currentSize--;
    siftDown(0, currentSize-1);      //自上向下调整为堆
    return true;                             //返回最小元素
};
```


堆
插入
新元素放到表尾（堆底）
根据大/小根堆 的要求，新元素不断“上升”，直到无法继续上升为止
删除
被删除元素用表尾（堆底）元素替代
根据大/小根堆 的要求，替代元素不断“下坠”，直到无法继续下坠为止
关键字对比次数
每次“上升”调整只需对比关键字 1 次
每次“下坠”调整可能需要对比关键字 2 次，也可能只需对比 1 次
基本操作
i 的左孩子 ——2i
i 的右孩子 ——2i+1
i 的父节点 ——⌊i/2⌋

1. 最小堆的性质是：

A) 每个节点的值都大于等于其子节点的值

B) 每个节点的值都小于等于其子节点的值

C) 左子节点的值总是小于右子节点的值

D) 根节点的值是最小的，其他节点无序

2. 在最小堆中插入新元素时，首先将新元素：

A) 放在根节点位置

B) 放在堆的最后一个位置

C) 与根节点比较并交换

D) 随机选择一个位置

3. 从最小堆中删除根节点后，通常用哪个元素来替换根节点?

A) 左子节点

B) 右子节点

C) 最后一个元素

D) 父节点

4. 构建包含n个元素的最小堆的时间复杂度是：

A) O(1)

B) O(n)

C) O(n log n)

D) O(log n)

1. 对最小堆进行堆化操作时，需要比较父节点与_____个子节点的值。

2. 数组 [3, 8, 2, 1, 9, 5] 构建成最小堆的结果是 _____。

3. 在最小堆中插入元素时，需要从下往上进行 _____ 操作。

4. 从最小堆中删除根节点后，需要从上往下进行 _____ 操作。

5. 包含10个元素的最小堆，高度为 _____。

6. 最小堆中第i个节点的父节点索引是 _____。

7. 对数组 [7, 3, 9, 2, 5] 进行原地建堆，第一次堆化时处理的非叶子节点是 _____。